



Практические занятия  
по дисциплине  
«Разработка веб-интерфейсного скрипта»

## Оглавление

|                                                                                           |     |
|-------------------------------------------------------------------------------------------|-----|
| Введение                                                                                  | 3   |
| Практическое занятие №1. Введение в JavaScript                                            | 4   |
| Практическое занятие №2. Базовый синтаксис JavaScript                                     | 15  |
| Практическое занятие №3. Циклы и функции. Их применение                                   | 33  |
| Практическое занятие №4. Структуры данных: объекты и массивы                              | 41  |
| Практическое занятие №5. Формы, элементы управления                                       | 51  |
| Практическое занятие №6. Формат JSON                                                      | 77  |
| Практическое занятие №7. Обмен данными в JavaScript через JSON                            | 82  |
| Практическое занятие №8. DOM. Управление DOM.                                             | 88  |
| Практическое занятие №9. Базовый синтаксис jQuery. Селекторы, события, эффекты            | 97  |
| Практическое занятие №10. Манипуляции DOM jQuery.                                         | 106 |
| Практическое занятие №11. Создание анимаций.                                              | 112 |
| Практическое занятие №12. Прокрутка, диаграммы, макеты, редакторы текста. Плагины jQuery. | 123 |

## Введение

Сегодняшний мир веб-сайтов трудно представить без языка JavaScript. JavaScript — это то, что делает живыми веб-страницы, которые мы каждый день просматриваем в своем веб-браузере.

Благодаря курсу «Разработка веб-интерфейсного скрипта», студенты освоят основы языка JavaScript, функции и объектно-ориентированные функции JavaScript, освоят основные методы программирования на языке JavaScript, а также методы отладки кода Javascript, представление объектов в формате JSON (формат обмена данными).

Научатся умело использовать JavaScript для разработки интерактивных страниц и эффектов, узнают, что такое DOM и как с ним работать, получат навыки использования базовых методов арифметики и встроенных функции, для достижения взаимодействия данных.

Овладеют навыками разработки интерактивных страниц с использованием селекторов jQuery, манипуляции DOM в jQuery, прокрутки, диаграмм, макетов, текстовых редакторов и плагинов для пользовательского интерфейса, событий и анимации jQuery.

Данный курс состоит из двух частей: программирование на JavaScript, которое является основой, и программирование на jQuery, которое является усовершенствованием.

Благодаря комплексному обучению, студенты освоят применение технологии JavaScript и легкого фреймворка JQuery в учебных проектах для достижения интерактивной веб-разработки и улучшения практических способностей веб-разработки.

# Практическое занятие №1. Введение в JavaScript

## ЦЕЛЬ РАБОТЫ

1. Познакомиться с языком программирования JavaScript. Изучить способы подключения файла JavaScript;
2. Вспомнить основы HTML и CSS;
3. Закрепить изученный материал путем самостоятельного написания первой программы на JavaScript.

## ХОД РАБОТЫ

Создадим нашу первую программу на языке JavaScript. Для написания и тестирования программ на языке JavaScript нам потребуются две вещи: текстовый редактор и веб-браузер.

В качестве текстового редактора можно использовать следующие редакторы - Sublime Text, Visual Studio Code, Notepad++ и другие. Так же в качестве редактора можно использовать приложение Блокнот. В рамках курса «Разработка веб-интерфейсного скрипта» мы будем ориентироваться на бесплатный и многофункциональный текстовый редактор Visual Studio Code, поскольку он является наиболее популярным.

В качестве браузера также можно взять последние версии любого предпочтительного веб-браузера. В настоящем руководстве я буду преимущественно ориентироваться на Google Chrome.

Для начала определим для нашего приложения какой-нибудь каталог. Например, создадим папку app. В этой папке создадим файл под названием lab1.html. То есть данный файл будет представлять веб-страницу с кодом HTML.

```
<!DOCTYPE HTML>
<html>
<body>
  <script>
    alert('Hello world!');
```

```
</script>
</body>
</html>
```

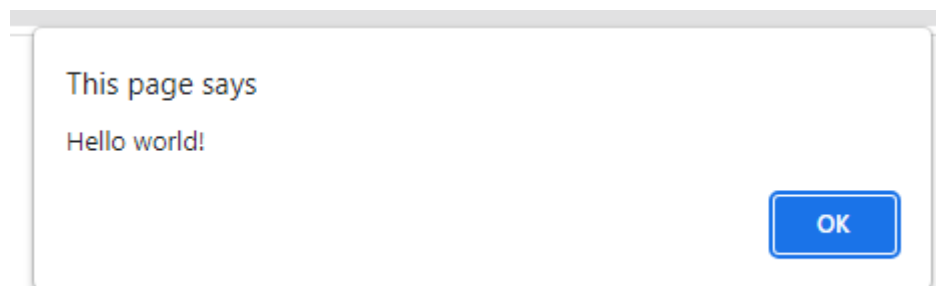


Рисунок 1 - Результат работы скрипта.

Как правило, только простейшие скрипты помещаются в HTML. Более сложные выделяются в отдельные файлы. Польза отдельных файлов в том, что браузер загрузит скрипт отдельно и сможет хранить его в кеше.

Другие страницы, которые подключают тот же скрипт, смогут брать его из кеша вместо повторной загрузки из сети. И таким образом файл будет загружаться с сервера только один раз. Это сокращает расход трафика и ускоряет загрузку страниц.

```
<!DOCTYPE HTML>
<html>
<body>
  <script src="lab1.js"></script>
</body>
</html>
```

```
alert('Hello world!');
```

Если атрибут `src` установлен, содержимое тега `script` будет игнорироваться. В одном теге `<script>` нельзя использовать одновременно атрибут `src` и код внутри. Нижеприведённый пример не работает:

```
<!DOCTYPE HTML>
```

```
<html>
<body>
  <script src="lab1.js">
    alert('New text');
  </script>
</body>
</html>
```

Нужно выбрать: либо внешний скрипт `<script src="...">`, либо обычный код внутри тега `<script>`. Вышеприведённый пример можно разделить на два скрипта:

```
<!DOCTYPE HTML>
<html>
<body>
  <script src="lab1.js"></script>
  <script>alert('New text')</script>
</body>
</html>
```

Помимо вышеописанного метода `alert()`, существуют другие методы для отображения диалогового окна в браузере: `prompt()` и `confirm()`.

Метод `alert()` предназначена для вывода в браузере предупреждающего модального диалогового окна с некоторым сообщением и кнопкой «ОК» - Рисунок 1. При его появлении дальнейшее выполнение кода страницы прекращается до тех пор, пока пользователь не закроет это окно. Кроме этого, оно также блокирует возможность взаимодействия пользователя с остальной частью страницы.

Метод `prompt()` предназначен для вывода диалогового окна с сообщением, текстовым полем для ввода данных и кнопками «ОК» и «Отмена». Это окно предназначено для запроса данных, которые пользователю нужно ввести в текстовое поле.

```
<!DOCTYPE HTML>
<html>
<body>
```

```
<script src="lab1.js"></script>
</body>
</html>
```

```
result = prompt('What is your name?');
alert(result)
```

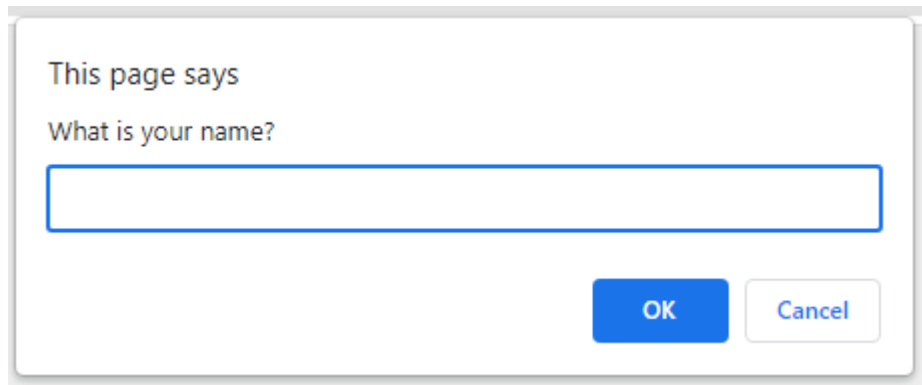


Рисунок 2 - Результат работы скрипта.

Запустите предложенный пример, и изучите результат. Попробуйте ввести имя или отменить операцию.

Метод *confirm()* применяется для вывода модального диалогового окна с сообщением и кнопками «ОК» и «Отмена». Оно обычно используется для запроса у пользователя разрешения на выполнение того или иного действия.

```
<!DOCTYPE HTML>
<html>
<body>
  <script src="lab1.js"></script>
</body>
</html>
```

```
result = confirm('Do you like JavaScript?');
alert(result)
```

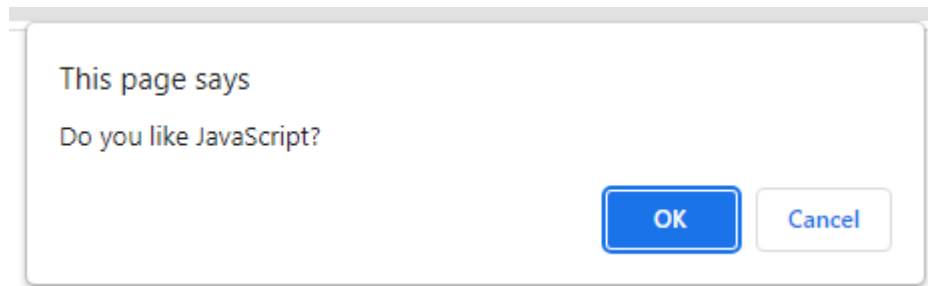


Рисунок 3 - Результат работы скрипта.

Выводить информацию можно не только с помощью диалоговых окон. Можно добавить на HTML страницу текст, путем вызова метода `document.write("<h1>My first programm!</h1>")`. С помощью *JavaScript* мы можем создавать на странице разные HTML элементы, это будет изучено подробнее в главе 9. Сейчас, на первом занятии нам понадобится только метод `write()`.

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Some text ...</p>
  <script src="lab1.js"></script>
  <p>... some text</p>
</body>
</html>
```

```
document.write("<h1>My first programm!</h1>");
```

Some text ...

# My first programm!

... some text

Рисунок 4 - Результат работы скрипта.

Как вы видите на рисунке 4, на страницу был добавлен текст, именно в том месте, где был вызван скрипт. Все дело в том, что когда браузер получает веб-страницу с кодом HTML и JavaScript, то он ее интерпретирует. Результат интерпретации в виде различных элементов - кнопок, полей ввода, текстовых блоков и т.д., мы видим перед собой в браузере. Интерпретация веб-страницы происходит последовательно сверху вниз.

Когда браузер встречает на веб-странице элемент `<script>` с кодом JavaScript, то вступает в действие встроенный интерпретатор JavaScript. И пока он не закончит свою работу, дальше интерпретация веб-страницы не идет.

В программе мы добавили на страницу `<h1>My first programm!</h1>` — это текст *My first programm!* обернутый в HTML тег `<h1>`. Давайте вспомним основные теги HTML и добавим их на страницу.

```
document.write("<h1>My first programm!</h1>");
document.write("<h2>My first programm!</h2>");
document.write("<h3>My first programm!</h3>");
document.write("<h4>My first programm!</h4>");
document.write("<h5>My first programm!</h5>");
document.write("<h6>My first programm!</h6>");

document.write("<hr>");

document.write("<p>My first programm!</p>");

document.write("<ol><li>My first programm!</li></ol>");
document.write("<ul><li>My first programm!</li></ul>");
```

```
document.write("<br>");

document.write("<b>My first programm!</b>");
document.write("<br>");
document.write("<i>My first programm!</i>");
document.write("<br>");
document.write("<del>My first programm!</del>");
```

Запустите предоставленный скрипт и посмотрите результат его работы в браузере.

HTML теги можно дополнять стилями, например изменить шрифт или цвет текста. Самостоятельно создайте скрипт, в котором на страницу добавляется отформатированный текст, например:

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Some text ...</p>
  <script src="lab1.js"></script>
  <p>... some text</p>
</body>
</html>
```

```
document.write("<h1>My first programm!</h1>");

document.write("<p style='font-size: 50px'>This is my first JavaScript  
programm!</p>");

document.write("<ul>");
document.write("<li style='color:blue'>HTML</li>");
document.write("<li style='color:green'>CSS</li>");
document.write("<li style='color:red'>JavaScript</li>");
document.write("</ul>");
```

Some text ...

## My first programm!

# This is my first JavaScript programm!

- HTML
- CSS
- JavaScript

... some text

Вы можете добавить в свой код комментарии, чтобы в будущем Вам или Вашим коллегам было удобнее разбираться в написанном вами скрипте. Комментарии не обрабатываются интерпретатором *JavaScript* и никак не учитываются в работе программы. Они предназначены для ориентации по коду, чтобы указать, что делает тот или иной код.

Комментарии могут быть однострочными, для которых используется двойной слеш - `//`. Кроме однострочных комментариев могут использоваться и многострочные. Такие комментарии заключаются между символами `/*текст комментария*/`. Например:

```
// comments
/* comments */
document.write("<h1>My first programm!</h1>");
```

Помимо комментариев в разработке может помочь следующая конструкция: `console.log(message)`. Это метод, предназначенный для печати в консоль браузера. Попробуйте добавить в существующий скрипт данный метод и посмотреть результат.

При написании скриптов иногда нужно увидеть промежуточный результат прямо в консоли браузера — это просто, удобно и не требует

никакой дополнительной логики для отображения. Чтобы открыть консоль используется горячая клавиша F12.

Попробуем написать простую программу, соединив весь изученный ранее материал. Программа должна запрашивать имя пользователя и выводить его на страницу и в консоль.

```
<!DOCTYPE HTML>
<html>
<body>
  <p>Some text ...</p>
  <script src="lab1.js"></script>
  <p>... some text</p>
</body>
</html>
```

```
result = prompt('What is your name?');

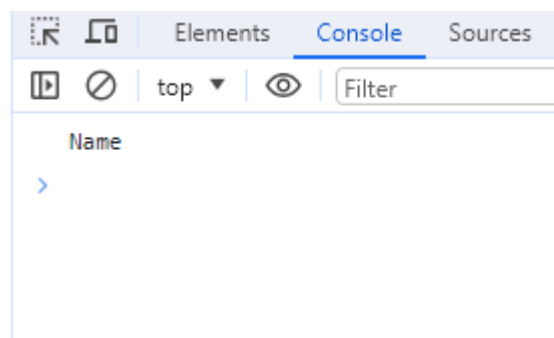
console.log(result);
document.write("<h1>" + result + "</h1>");
```

При запуске данного скрипта текст, который был введен пользователем, отразится на странице и в консоли браузера.

Some text ...

**Name**

... some text



Попробуем создать на странице кнопку.

```
<!DOCTYPE HTML>
<html>
<body>
  <button id="myButton">Click on me</button>
```

```
<script src="lab1.js"></script>
</body>
</html>
```

```
const button = document.getElementById('myButton');
button.onclick = () => {
  alert("Button is clicked");
};
```

Бывают ситуации, когда действие на странице нужно вызывать не по нажатию кнопки или другому действию пользователя, а по прошествии какого-то времени. Например, каждую минуту напоминать пользователю об оплате заказа.

Мы можем вызвать функцию не в данный момент, а позже, через заданный интервал времени. Это называется «планирование вызова». Для этого существуют два метода:

- *setTimeout* позволяет вызвать функцию один раз через определённый интервал времени.
- *setInterval* позволяет вызывать функцию регулярно, повторяя вызов через определённый интервал времени.

Эти методы не являются частью спецификации *JavaScript*. Но большинство сред выполнения JS-кода имеют внутренний планировщик и предоставляют доступ к этим методам.

```
<!DOCTYPE HTML>
<html>
<body>
  <button id="myButton">Click me!</button>
  <script src="script.js"></script>
</body>
</html>
```

```
setTimeout(() => alert('Hello!'), 5000);
```

Запустите самостоятельно скрипт и попробуйте изменить `5000` – количество миллисекунд на другое значение. Поменяйте метод `setTimeout` на `setInterval`. Сравните полученные результаты.

Для отмены выполнения действий необходимо вызвать `clearInterval/clearTimeout` со значением, которое возвращают методы `setInterval/setTimeout`. Например, если пользователь уже оплатил заказ, и разработчику уже нет необходимости напоминать об этом. Добавим в скрипт условие, что нужно прекратить вывод диалогового окна через *15 секунд*.

```
const interval = setInterval(() => alert('Hello!'), 5000);
setTimeout(() => clearInterval(interval), 15000);
```

Обратим внимание, что все методы планирования не гарантируют точную задержку. Например, таймер в браузере может замедляться по многим причинам:

- Перегружен процессор.
- Вкладка браузера в фоновом режиме.
- Работа ноутбука от аккумулятора.

Всё это может увеличивать минимальный интервал срабатывания таймера (и минимальную задержку) до 300 или даже 1000 мс в зависимости от браузера и настроек производительности ОС.

Вы познакомились с языком программирования JavaScript, изучили способы подключения файла JavaScript на HTML страницу. Так же вспомнили основы HTML и CSS. Научились вызывать диалоговые окна, создавать кнопки на странице и обрабатывать события нажатия на них, научились добавлять на веб-страницу текст и логировать действия в консоли разработчика. Теперь осталось закрепить изученный материал путем самостоятельного написания первой программы на JavaScript!

## ЗАДАНИЯ

1. Создайте простую HTML страницу, на которой выводится Ваше имя и номер группы. Добавьте стили к отображаемому на странице тексту;
2. Создайте отдельные страницы для следующих заданий. Добавьте на первую созданную страницу ссылки на страницы с другими заданиями;
3. Создайте скрипт, который выводит на страницу ваше имя и сообщение «Я изучаю JavaScript!» в диалоговом окне, при первом открытии страницы и после нажатия на кнопку.
4. Придумайте список вопросов для пользователя, например: Ваш любимый цвет? Ваше любимое животное? (не менее 5 вопросов). Реализуйте опрос по подготовленным вопросам в виде диалоговых окон. Результаты должны отображаться на странице в виде:
  - a. Вопрос 1: Ваше имя?
  - b. Ответ 1: Иван.
5. Для полученных ответов пользователя добавьте на страницу с опросом стили. Например, выделите все ответы курсивом и красным цветом:
  - a. Вопрос 1: Ваше имя?
  - b. Ответ 1: Иван.*
6. Создайте скрипт с использованием планирования вызова. Скрипт должен выводить на страницу диалоговое окно с вопросом, например вопрос «Какая сегодня погода?» каждые 2 секунды. Результат, ответ пользователя, должен выводиться на страницу или в консоль браузера.